Ders 6 Çalışma Özeti

Ders 6 Çalışma Özeti

Genel Konular

- Uygulama projesinden beklentiler: plan, gereksinim analizi, geliştirmenin bir kısmı, kontrol ve izleme.
- Maliyet tahmin yöntemlerinin tekrarı: bilirkişi takdiri, Delphi, analiz yöntemi, istatistik modeller.
- COCOMO, Putnam modeli ve "birleştirme şaşkınlığı".
- Yazılım ölçümü tekrarı: LOC/KLOC ve işlev noktası (Function Point).
- Makale ödevi: 15-20 dakikalık sunum ve IEEE formatında 3-4 sayfalık yazı.
- Proje ekip yapıları: demokratik, şeflik, hiyerarşik.
- Proje organizasyon yapıları: fonksiyonel, proje, matris organizasyon.
- Çevik (agile) yaklaşımlarda ekip yapısı.
- Proje zamanlaması ve yazılım proje planının içeriği.
- Sunum takvimi ve ara sınav hatırlatması.

Hocanın Özellikle Vurguladığı Kısımlar

- Uygulama projesinde yalnızca plan yapıp bırakmak yeterli değildir. Geliştirme sürecinde planın nasıl izlendiği görülmeli.
- Bitmiş bir projenin planı istenmiyor. Yeni planlanan veya prototip aşamasındaki bir proje olmalı.
- Kaynak kodu istenmiyor. Gereksinim analizi dokümanı (SRS) ve küçük bir modülün tasarım başlangıcı bekleniyor.
- En az iki kilometre taşında planlanan ile gerçekleşen karşılaştırılmalı. Plan ve riskler buna göre revize edilmeli.
- Proje yönetimi yol haritası: başlangıç, planlama, yürütme (project running), kontrol ve izleme, kapanış.
- Mühendislikte iki temel yetenek vardır: tasarlama ve ölçme.
- Bir iş yükünü kısa sürede bitirmek için çok sayıda kişi yığmak işe yaramaz. Yazılım geliştirme belli bir süreçten geçmelidir.
- Şeflik yapısı yaklaşık 10 kişiden büyük ekipler için önerilmez. Büyük ekipler için hiyerarşik yapı uygundur.
- Her ortama uygun tek bir organizasyon tipi yoktur. Seçim işe, çalışma ortamına ve kurumun amaçlarına göre yapılır.
- Çevik konusunda makale seçenler temel bilgilerin ötesine geçmeli, konuyu daha özel bir yöne doğru incelemeli.

Kısa Tekrar Notları

- Temel COCOMO: $E = a \cdot \text{KLOC}^{b}$, $T = c \cdot E^{d}$. Üç aşama: büyüklük tahmini, iş gücü, süre.
- COCOMO düzeyleri: temel, orta (düzeltme faktörü eklenir), ileri.
- Slayttaki örnek: 10 KLOC için $E = 3{,}6 \times 10^{1{,}20} \approx 57$ kişi-ay.
- Birleştirme şaşkınlığı: modüller için ideal süreler yukarı doğru toplanınca beklenenden çok uzun süre çıkması (örneğin 2,5 yıl).
- Düzeltilmiş işlev noktası: $FP = \text{ham FP} \times (0{,}65 + 0{,}01 \sum_{i=1}^{14} F_i)$.
- Albrecht yöntemi: büyüklük birimi olarak KLOC yerine işlev noktasını kullanır.

- Ekip yapıları:
  - Demokratik: herkes eşit, kararlar uzlaşmayla. En çok 8-10 kişi. Yavaş işler.
  - Şeflik: kararları şef verir. Yaklaşık 5, en çok 10 kişi. Hızlıdır ama kalite şefin yeteneğine bağlıdır.
  - Hiyerarşik: proje yöneticisi, ekip şefleri ve çalışanlar. Büyük ekipler için uygun.
- Organizasyon yapıları:
  - Fonksiyonel: uzmanlık alanlarına göre bölümler.
  - Proje: modüllere göre ekipler, her ekipte gerekli tüm uzmanlıklar.
  - Matris: fonksiyonel ve proje yapısının birleşimi. Çalışan iki yöneticiye raporlar.
- Makale ödevi: 15-20 dakikalık sunum (9-11. haftalar) ve dönem sonunda IEEE formatında 3-4 sayfalık yazı.

Detaylı Açıklamalar

Uygulama projesi hakkında sorular

- Hoca uygulama projesi önerilerinin hiçbirini reddetmedi. Amacı herkesin konusunu önceden bilmekti.
- Bir öğrenci kaynak kodu gerekip gerekmediğini sordu. Cevap: kaynak kodu gerekmiyor.
- Bir öğrenci sıfırdan planlanan bir projenin yalnızca planını sunmanın yeterli olup olmayacağını sordu. Cevap: biraz geliştirme aşamasına da geçilmeli.
  - Proje yönetim sürecinin tamamı görülmek isteniyor: başlangıç, planlama, kontrol ve izleme, kapanış.
- Bir öğrenci geliştirme için ekran görüntüsü veya anlatımın yeterli olup olmayacağını sordu. Hocanın beklentisi:
  - Gereksinim analizine girilmiş ve bir gereksinim analizi dokümanı hazırlanmış olmalı.
  - Tasarımda en az küçük bir modülün başlangıcı yapılmalı.
  - En az iki kilometre taşı geçildikten sonra planlanan ile gerçekleşen arasındaki farklar izlenmeli. Riskler güncellenmeli, plan revize edilmeli.
- Gantt diyagramında planlanan ile gerçekleşeni karşılaştırma özelliği var. Öğrenciler ayarlarını deneyebilir.
- Hoca planlamayla ilgili bir örnek çalışmayı sonraki hafta göstermeyi önerdi.

Maliyet tahmin yöntemlerinin tekrarı

- Delphi yöntemi, bilirkişi takdirinin çok uzmanlı hâlidir. Sorular dağıtılır ve ortak karara yaklaşılana kadar tekrarlanır. Her türlü grup kararında kullanılabilir.
- Analiz yöntemi: proje alt parçalara bölünür ve alttan yukarı doğru tahmin yapılır.
  - Ürüne göre: modüller ve alt parçalar.
  - Üretime göre: proje yönetimi, tasarım, kodlama, birim test, test gibi safhalar.
- İstatistik modeller: doğrudan ölçülemeyen bir değer, ölçülebilen bağımsız bir değişkenle kestirilir. Yazılımda genellikle üslü fonksiyonlar kullanılır. Analiz yöntemiyle birlikte uygulanır.

COCOMO

- COCOMO (Constructive Cost Model), Barry Boehm'in modelidir. En yaygın maliyet tahmin modelidir.
- Temel COCOMO'da büyüklük birimi KLOC'tur. a ve b katsayıları çok sayıda proje üzerinde yapılan denemelerle bulunmuştur.
- T: belirli bir iş yükünün etkin olarak ne kadar sürede bitirilebileceği.
- Uygulama adımları:
  1. Ürünün alt sistemleri ve modülleri tanımlanır.
  2. Her modülün büyüklüğü tahmin edilir.
  3. Orta COCOMO kullanılıyorsa ürün, donanım ve personel özelliklerine göre düzeltme faktörleri alınır.
  4. Her modül için iş hacmi ve süre hesaplanır.
  5. Sonuçlar toplanarak toplam iş gücü ve süre bulunur.
- Birleştirme şaşkınlığı ve düzeltilmesi:

  - Yukarı doğru toplanan ideal süre, müşterinin veya yönetimin beklediği süreden çok uzun çıkabilir.
  - Düzeltmek için kaynaklar daha verimli kullanılır, daha sıkışık bir çalışma düzenine geçilir, bazı gereksinimler revize edilir.
  - Hesaplanan değer, Delphi ile takdir edilen değerle karşılaştırılır ve fark giderilmeye çalışılır.
- Fayda-maliyet değerlendirmesi için duyarlılık analizi yapılır. Planlama ve analiz giderleri eklenir.
- Hocanın kendi çalışmaları: önce yapay sinir ağlarıyla bir maliyet tahmin modeli geliştirildi. Yaklaşık bir buçuk yıl önce biten bir çalışmada bu model çevik projeler için revize edildi.

Putnam ve diğer modeller

- Putnam modeli (altyazıda "Putnam-Rayli"): KLOC, teknoloji katsayısı ve geliştirme süresiyle iş gücünü hesaplar. Fonksiyonu COCOMO'dan farklıdır.
- COCOMO ve Putnam büyüklük birimi olarak KLOC kullanır.
- Albrecht yöntemi (işlev noktası maliyet yöntemi) KLOC yerine işlev noktasını kullanır.
- Tüm bu modellerde önce projenin büyüklüğü tahmin edilir. Sonra iş gücüne, iş gücünden de süreye geçilir.

Yazılım ölçümü

- Yazılım mühendisliği, yazılım ürünü oluşturmak için mühendislik yaklaşımını uygulayan teknikler bütünüdür. Hoca bu tanımı önceki dönemin test dersinde de vermişti.
- Mühendislik yaklaşımının her adımında (yönetim, maliyet hesabı, planlama, modelleme, analiz, tasarım, gerçekleştirme) ölçme vardır.
- Doğrudan ölçüm: LOC ve KLOC. Satırlar sayıldığı için en kolay yöntemdir.
  - Zayıf yanı: etkin bir algoritma daha az satır üretir, farklı bir dil daha çok satır üretebilir.
  - Diller arası farkı gidermek için dönüşüm tabloları vardır. Ama algoritma etkinliğinden gelen fark giderilemez.
- Dolaylı ölçüm: işlev, nitelik, karmaşıklık, etkinlik ve güvenilirlik gibi özelliklere dayanır. Örneğin belge, dosya ve ekran sayıları kullanılır.
- İşlev noktası hesabı:
  - Sayılan öğeler: kullanıcı girdisi, çıktı, sorgu, dosya, dış arabirim sayısı.
  - Her sayı, basit, orta veya karmaşık olmasına göre ağırlık faktörüyle çarpılır. Toplamları ham işlev noktasıdır.
  - 14 genel özellik sorusu 0-5 arası puanlanır. Örnekler: yedekleme ve geri yükleme (backup recovery), iletişim altyapısı, dağıtık özellikler, kritik performans, sistem yükü, çevrim içi veri girişi.
  - Slayttaki örnekte iki özelliğe 5, yedekleme ve geri yüklemeye 4 verilmiş. Toplam $\sum F_i = 32$.
- İşlev noktası da LOC gibi şu ölçümlerde kullanılır:
  - Verimlilik: bu büyüklükte bir iş kaç kişi-ayda geliştirilir.
  - Kalite: işlev noktası başına hata.
  - Gider: işlev noktası başına maliyet.
  - Belgeleme: işlev noktası başına belge.
- LOC ve FP karşılaştırması:
  - LOC programlama diline bağlıdır. FP dilden bağımsızdır; dosya, ekran, girdi ve çıktı sayılarıyla dışarıdan hesaplanır.
  - LOC iyi tasarlanmış kısa yazılımları yeterince değerlendiremez.
  - FP özneldir ve verisi zor toplanır. LOC'un en büyük faydası doğrudan fiziksel bir ölçü olmasıdır.
  - Çalışmaya hangisi uygunsa o kullanılır.

Makale ödevi

- Hoca makale önerilerinin çoğuna cevap verdi.

- Bir öğrencinin önerdiği makale yazılımcı hatalarını insan davranışı ve kişilik özellikleri açısından inceliyor (IEEE Transactions on Software Engineering). Hoca bunu uygun buldu.
  - Hoca bu konuda grubunda daha önce çalışmalar yapıldığını söyledi.
- Bu yıl önerilen konuların çoğu çevik yaklaşımlar ve Scrum üzerineydi ve çakıştı. Hoca bunları kabul etti ama daha özel ve yeni yönlere doğru incelenmesini istedi.
- Ödevin iki parçası var:
  1. 15-20 dakikalık sunum. Sunumlar 9. haftada başlayacak; kimse hazır değilse 10. haftada.
  2. Dersin son haftasında online sisteme yüklenecek, IEEE formatında 3-4 sayfalık yazı. Bir büyük makalenin özeti veya birden fazla bildirinin literatür özeti (review) olacak. Kendi cümlelerle yazılmalı.
- Makale sunumlarından sonra uygulama projesi sunumları yapılacak.

Proje ekip yapıları

- Plan aşamasında şimdiye kadar yapılanlar: risk analizi, varsayım ve kısıtlar, aktiviteler ve ilişkileri, Gantt ve PERT ile zaman planı, kritik yol, maliyet tahmini ve bütçe. Geriye organizasyon kalıyor.
- **Demokratik ekip yapısı:**
  - Tüm üyeler eşit ve aynı yetkiye sahip.
  - Amaçlar ve yöntemler uzlaşmayla belirlenir.
  - İletişimi sağlamak için bir koordinatör seçilebilir.
  - En çok 8-10 kişi olabilir. Herkes konuştuğu için işler yavaş yürür.
- **Şeflik ekip yapısı:**
  - Tasarım, geliştirme ve teknik konularda kararları şef verir. Görev dağıtımını şef yapar.
  - Yaklaşık 5 kişilik küçük bir yazılım evi gibi. Örneğin deneyimli bir mühendisin kurduğu 5-7 kişilik şirket.
  - Hızlı yürür ama kararların doğruluğu şefin yeteneğine bağlıdır.
  - 10 kişiden büyük ekipler için önerilmez.
- **Hiyerarşik ekip yapısı:**
  - Bir proje yöneticisine bağlı birkaç ekip vardır.
  - Proje yöneticisi görevleri belirler, çalışmaları izler ve denetler, iş yüklerini dengeler.
  - Ekip şefleri kendi alt sistem veya modüllerinden sorumludur.
  - Proje yöneticisi her işe kendisi karar vermez.

Proje organizasyon yapıları

- **Fonksiyonel organizasyon:**
  - Çalışanlar uzmanlık alanlarına göre bölümlere ayrılır: sistem analizi, programlama, pazarlama, bakım gibi.
  - Her grubun başı projenin kendi kısmından sorumludur.
  - Çok büyük projelerde ve organizasyonlarda kullanılır. Küçük projelerde hantal kalır.
- **Proje organizasyonu:**
  - Sistem ana parçalara ayrılır. Her parça için gereken tüm uzmanlardan oluşan bir birim kurulur. Örnek: genel uygulamalar sistemi içinde muhasebe modülü için ayrı bir birim.
  - Slayttaki örnek:

```
Proje Yöneticisi
├── Destek: bakım yöneticisi, kalite sorumlusu, satış sonrası destek uzmanı
├── Stok Kontrol Ekip Lideri ── sistem analisti, programcı
├── Finans Ekip Lideri ── sistem analisti, programcılar
├── Yönetim Bilgi Sistemleri Ekibi ── sistem analisti, programcılar
└── Müşteri Ekranları Ekibi
```

- Ekipler bazen paralel, bazen birbirini izleyerek çalışır.
- **Matris organizasyon:**
  - Ağaç yapısı yerine tablo yapısı vardır. Yetki birden fazla kişiye dağıtılmıştır.
  - Fonksiyonel ve proje organizasyonunun üst üste konmuş hâli gibidir.

  - Sürekli yöneticiler: yazılım mühendisliği lideri, sistem analistleri lideri, programcılar lideri. Bunlar genel müdüre bağlıdır.
  - Projeler (A projesi, B projesi) ayrıca proje yöneticilerine sahiptir.
  - A projesindeki bir programcıyı hem programcılar lideri hem de A projesinin yöneticisi kontrol eder.

```
                  A Projesi Yöneticisi   B Projesi Yöneticisi
Yazılım Müh. Lideri    mühendis              mühendis
Sistem Analist Lideri  analist               analist
Programcılar Lideri    programcı             programcı
```

Çevik yaklaşımlarda ekip yapısı

- Klasik ekip ve organizasyon yapıları çevik yaklaşımlarla tam örtüşmez.
- Scrum'da proje yöneticisi yoktur, herkes birbirini kontrol eder.
- Yine de şirkette programcıları yöneten, başarı grafiklerini isteyen ve izinlerini veren bir birim olabilir.
- Çevik ekipte herkes birbirinin yaptığından haberdardır. Kimse "ben programcıyım, gerisini bilmiyorum" demez.
- Projede alt başlık açılmaz. Ürün, belirli fonksiyonları yerine getiren ve alt modüllerde çalışanlardan oluşan genel grubun ürünüdür.

Proje zamanlaması ve proje planı

- Proje adımları ve onay noktaları kesin tarihlerle planlanmalı. Durak noktalarında onaya sunulmalı ve süre bitiminde teslim edilmelidir.
- Zamanlama PERT, CPM ve Gantt çizelgesiyle gösterilir.
  - PERT istatistiksel fonksiyonlarla her işlem için en olası süreyi kestirir. CPM bunu yapmaz, çünkü süreleri belli kabul eder.
  - Yazılımda planlama ve gereksinim analizi tahmin edilenden uzun sürebilir. Bu yüzden PERT'in başlama-bitirme aralığı hesabı önemlidir.
- Proje planı, yazılım mühendisinin bilgi ve deneyimine dayanarak hazırlanır. Görev sırasında plan gözden geçirilir ve düzeltilir.
- Proje planının içeriği:
  - Projenin kapsamı.
  - Yazılım geliştirme süreç modeli. WBS'den önce seçilir.
  - Risk analizi.
  - Alt aktiviteler (WBS) ve aralarındaki ilişkiler.
  - İş gücünün belirlenmesi (maliyet tahmin yöntemleriyle).
  - Görev ataması: önce rastgele atanıp yoğunluğa göre dengelenebilir. Bunun için endüstri mühendisliği yöntemleri ve çalışmaları vardır.
  - Bütçe.
  - Kalite planı.
  - Yazılım gereksinim spesifikasyonu (SRS).

Sunum takvimi ve duyurular

- Makale sunumları 9, 10 ve 11. haftalarda tamamlanacak. Sonra uygulama projesi sunumları gelecek.
- Bir öğrenci 9. hafta için gönüllü oldu. Makalesinde bir anket çalışması var. Hoca konuyu ilginç buldu ve anketin sektör desteğiyle geliştirilmesini önerdi.
- Bir öğrenci 10. haftayı istemişti.
- Bir öğrenci anket çalışması ve seminer dersi yüzünden son haftaları istedi. Bir anket yapmayı düşünüyor.
- Bir öğrenci 9. haftada işi yoğun olduğu için 10 veya 11. haftayı istedi.
- Kesin takvim sonraki hafta herkese sorularak yazılacak.
- 7. hafta ders yapılacak. 8. hafta ara sınav yapılacak. 9. haftada hocanın anlatacağı bazı konular da var.
- Proje dokümanları için online sistemde yükleme alanı açılacak.

Ders sonrası notlar

- Altyazıdaki “Ecail”, “Eca ile”, “Ecel” kelimeleri çevik (agile) yaklaşımları ifade ediyor.
- Altyazıdaki “Kokomo” COCOMO, “Bohem/BOEM” Boehm, “Kalog/Karlok” KLOC, “albrekt” Albrecht olmalı.
- Hoca Boehm’i “yazılım mühendisliğinin kurucusu” olarak andı. Boehm yazılım ekonomisi ve maliyet tahmini alanının öncüsüdür. “Yazılım mühendisliği” terimi ve alanın kuruluşu genelde 1968 NATO konferansıyla ilişkilendirilir.
- Hoca “Function Point işlemsel olmayan dillere kolayca uyarlanamaz” dedi. Literatürde bu zayıflık LOC için sayılır; 5. dersteki slaytta da böyledir. FP’nin üstünlüklerinden biri dilden bağımsız olmasıdır.
- Hoca PERT ve CPM için “aynı işi yapıyorlar” dedi. İkisi de kritik yolu bulur. Fark şudur: PERT belirsiz süreler için üç tahminle çalışır, CPM süreleri kesin kabul eder.
- COCOMO örneğindeki formülde slaytta üsler normal yazılmış. Hoca 1,20 değerinin üs olduğunu belirtti.

Not: İsterseniz bu kayıtları NotebookLM gibi bir araca atıp daha istifadeli hale getirebilirsiniz.